PipedPeer: Zero-Configuration Distributed Computing via Recursive, Decentralised Coordination

Yateen Vaviya
Department of Computer Engineering
Sardar Patel Institute of Technology
Mumbai, Maharashtra, India
yateen.vaviya23@spit.ac.in

Vinayak Yadav
Department of Computer Engineering
Sardar Patel Institute of Technology
Mumbai, Maharashtra, India
vinayak.yadav23@spit.ac.in

Sahil Gupta
Department of Computer Engineering
Sardar Patel Institute of Technology
Mumbai, Maharashtra, India
sahil.gupta23@spit.ac.in

***Abstract*—Distributed computing frameworks need a cluster that somebody has already configured, a program rewritten against their API, and machines that can reach one another. Those costs come before any speedup, and they put the idle machines a small team owns out of reach, particularly when the machines sit behind different routers. We present PipedPeer, which runs an unmodified Python script on other machines with no manual installation: one binary whose set-up command installs the rest. The script's environment is built as a content-addressed Nix closure against a pinned package set, so the same script yields the same store path on every machine. A worker that holds the environment ships nothing, and one that holds part of it receives only the missing store paths. The parallel primitives the script already uses are substituted at import time. Parallel maps are divided among peers by a makespan solver over measured throughput, and numerical work moves only when a cost model finds moving it cheaper than computing it. Machines on different networks are joined with one command: an introducer that carries no data exchanges addresses, and the machines punch through their routers, with a birthday-collision fallback for the hardest pair of NATs, onto direct QUIC links authenticated by per-node Ed25519 keys. No relay carries job data. In a three-worker container lab, a repeat submission took 0.83 s against 23.4 s for the first. Interception costs $1.24\times$ the start-up of a job that imports nothing heavy, and about 0.12 s on a job it declines. A run can be stopped on every machine with one interrupt, or detached and reattached, and a training ring continues when a rank dies and grows when a machine returns. 822 of 829 tests pass with 0 failing, and a mutation audit breaking 95 features is caught in every case. Measurements recorded between real machines during development are reported with their provenance and are not claimed beyond it.**

***Index Terms*—distributed computing, peer-to-peer, NAT traversal, transparent interception, reproducible environments, content-addressed caching, cost-based scheduling**

I. Introduction

The gap between running `python script.py` and running the same work on several machines is large in operational effort and often small in the computation involved. Apache Spark [1], Ray [2] and Dask [3] all presuppose a configured cluster with a head node, matching software on every worker, a scheduler process, and a network on which every machine reaches every other. Remote execution over `ssh` avoids the set-up but provides no environment management, placement, isolation or fault tolerance, and it still needs a reachable address.

Three obstacles are usually treated separately. The *environment* obstacle: a remote machine must be made to match the local one. The *code* obstacle: using more than one machine normally requires rewriting the program. The *reachability* obstacle: machines behind different home or campus routers accept no inbound connections, and the usual answers — a VPN, port forwarding, or a relay — are either configuration or a bottleneck. PipedPeer removes all three. Its contributions are:

1) **Peer-to-peer closure distribution keyed and diffed by store path.** Content-addressed store paths and substitution are Nix's [5]; we add pushing signed closures between peers, sending only the store paths a receiver lacks, and using a runnable-closure check to decide which peers may receive distributed work (Section III-B).
2) **Transparent interception with measured work sharing.** Parallel primitives are substituted at import time; parallel maps are divided by a makespan solver over measured peer rates, and numerical work is gated by a cost model (Sections III-D, IV).
3) **Joining across networks without a relay.** An introducer that only exchanges addresses, router port mapping, a candidate race on one shared socket, a birthday-collision fallback, and authenticated QUIC links that the rest of the system sees as ordinary nodes (Section III-C).
4) **Placement without a scheduler.** Hard filters and a bounded score evaluated in the submitting client, with a three-phase lease and per-machine memory budgets (Section III-A).
5) **A correctness-preserving fallback on every distributed path**, with a test suite audited by disabling 95 features in turn and requiring the tests to notice each (Section V).

II. Related Work

Spark [1], Ray [2] and Dask [3] are mature and considerably more capable at scale. Ray ships a drop-in `multiprocessing.Pool` and a `joblib` backend, and Modin [10] provides distributed pandas behind the pandas API; each requires changing an import and a running cluster. Here no line of the program changes, because substitution happens in `sitecustomize` before the first user statement, and distribution is gated per call. We are not aware of prior work combining import-time substitution of unmodified code with a per-call cost gate. BOINC [4] pioneered volunteer computing but is a platform for specific projects rather than a general executor.

Our cross-network mechanism follows established practice. Reflexive addresses are discovered as in STUN [13]; port mappings are requested with PCP [15] and NAT-PMP [14]; and simultaneous UDP probing from the socket that will carry the connection is the hole-punching technique of Ford et al. [16]. Links are QUIC [12].

Edge-computing offloading addresses the decision our cost model makes. Deng et al. [6] compare transmission against local computation; Yong et al. [7] show partial offloading to beat all-or-nothing, the regime our interception operates in; learned policies [8] exceed a deterministic rule at the price of a training phase; and topology-aware placement [9] outperforms scalar scoring, a limitation our whole-job placement shares. Bergström [11] surveys abstractions that hide failure from application code, the property Section V must provide for scripts that contain no failure handling.

III. System Design

Every machine runs one daemon; the command-line client is short-lived. The coordinator is a library linked into the client, so there is no scheduler process and no head node. A machine is a submitter and a worker at once. On one network a participant joins by running the binary; across networks, with `pipedpeer join` and an introducer's address.

A. Placement

Candidates come from known peers, LAN probes, internet-linked peers and an optional registry. Five hard filters run in order — architecture, GPU if required, and free memory, cores and VRAM — and survivors are scored:

$$S = \Big(1 - \tfrac{u_{\text{cpu}}}{200} - \tfrac{u_{\text{mem}}}{200} - 0.05\, n_{\text{jobs}} + 0.10(\hat{c} - \tfrac{1}{2}) + 0.06(\hat{f} - \tfrac{1}{2}) + 0.10(\hat{m} - \tfrac{1}{2}) + 0.10(\hat{r} - \tfrac{1}{2})\Big) \cdot H \quad (1)$$

where u are utilisations in percent, n_{jobs} the running jobs, $\hat{c}, \hat{f}, \hat{m}, \hat{r}$ core count, clock, free memory and free-core ratio normalised onto $[0, 1]$, and H a health score. Load dominates hardware (the load terms can subtract 1.0, the hardware terms move the score by at most 0.36), and a missing metric normalises to the neutral 0.5. Free memory is what a node can *grant*: where a user `systemd` is available each daemon runs under a cgroup ceiling of half the machine, and daemons on one machine subtract one another's reservations.

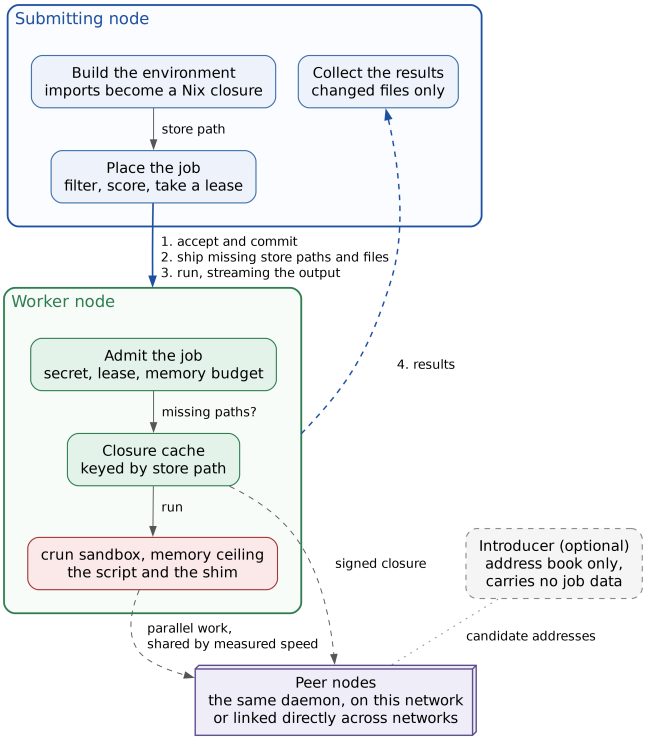


Fig. 1: System design. Both roles are the same binary; peers on other networks are reached over direct links that the introducer helps establish but never carries.

Work is reserved by a three-phase lease. The concurrency and memory checks and the insertion occur under one lock, and the concurrency limit counts reserved and running leases together. A submitter that dies stops renewing and its slot is reclaimed; a worker that dies drops out of the healthy set and its job is rescheduled with back-off.

B. Environment Reproduction

The client scans imports, resolves them to packages, and builds a Nix closure from a flake that pins an *exact* package-set revision. A branch would resolve differently on different days and change the store path; an exact pin makes the store path a stable name for an environment across machines and time. An import the mapping table does not know is looked up as the package of the same name in the pinned set, the scan follows the script's own modules, and a `requirements.txt` beside the script is honoured; an import still unresolved is reported before the run rather than dropped.

Before uploading, the client asks whether the target can already run the closure. If not, it lists the closure's store paths and asks which are missing, sending only those; adding one package to an environment a peer already held sent 7 of 43 paths, 206.0 MB reduced to 28.6 MB. After an upload the worker pushes the closure to peers that cannot yet run it, since only those that can are offered distributed work. Between

daemons the push is a signed binary cache produced and imported by Nix itself, signed with the sending node's key, so the receiver's signature policy applies. The archive names its signer, whose key the receiver checks against the signer's published identity before trusting it for that import; on a multi-user Nix, whose daemon ignores keys passed per command, a root-owned helper installed at set-up can only add or remove keys of the cluster's form. Caches are pruned least-recently-used.

C. Joining Across Networks

An introducer keeps an address book, keyed by a hash of the cluster's shared secret, and forwards one punch request between two nodes; it carries no job data. Each node offers its LAN addresses, a port mapped on its router (published only if the router's external address matches what the introducer observed, which rules out a router behind a second NAT, once the reflexive address is known), and its reflexive address. Probes, introducer messages and QUIC share one UDP socket, because a router's mapping belongs to the socket that opened it. Both sides probe every candidate pair every 250 ms for 15 s. When one side's NAT allocates a new port per destination there is no address to publish; that side opens 256 sockets toward the peer while the other sends 600 probes to random ports, and the socket that is hit is kept. The winning path carries QUIC with a hello signed by each node's Ed25519 key, checked against the node it meant to reach; the hello signs the session's exported keying material [17] and proves the cluster secret by an HMAC over it, so it cannot be replayed on another session. The link is exposed locally as a loopback forwarder registered as an ordinary node, so placement, transfer and gradient exchange run over it unchanged. A peer with no path is reported with its reason and routed around, never relayed. A container lab with cone and symmetric translators found four defects that kept such links from forming or lasting, among them a collision-found socket that stopped listening, and kept a read deadline, once adopted; all are fixed.

The daemon API requires the cluster's shared secret on every route when one is configured, sends it only to hosts that answer a challenge with an HMAC, and serves pinned TLS and HTTP on one port. These protections are optional and partial: without a secret the daemon is open, TLS is trust on first use and can fall back to plain text, and the set of keys trusted to sign closures is not yet bound to an authenticated identity.

D. Transparent Interception

A Python module embedded in the binary is appended to the workspace archive and placed on the interpreter's path, so it is imported before the user's first line; libraries are patched on first import. Substituted primitives are parallel maps, pandas grouping, joining and chunked reading, NumPy matrix products and decompositions, `joblib`, and data-parallel training.

A parallel-map kernel travels as *source* when it is a plain module-level function, including one defined in `__main__`, or *by value* through cloudpickle when it is a lambda or closure. A chunk is divided among peers by a makespan solver: each device is modelled as a set-up time plus a rate, fitted from the parts it has returned, and a device whose set-up alone exceeds the makespan is declined. On a lab cluster limited to 8, 2 and 1 CPUs, a headcount split was 1.6× slower than not distributing; the measured split declined the 1-CPU worker and took 2.87 s against 3.61 s.

Grouping and joining hash rows by key, so each key is complete on one node and any aggregation is exact in value with no combiner; row order of joins is not preserved. For data-parallel training, ranks post gradients to the lead node's daemon port rather than building a rank-to-rank mesh, and the daemon returns one sample-weighted average, $\sum_i n_i g_i / \sum_i n_i$, in half precision by default. Returning every rank's gradient instead had made two machines 21 times slower than one. Samples are assigned to ranks by a weighted rendezvous hash [18], so when a rank dies the lead daemon stops waiting for it and the survivors take its share at the next epoch boundary, while over 95% of each survivor's samples stay in place; the ring still depends on rank 0's machine, whose daemon holds the synchronisation. A rank whose machine returns, or a new eligible node, joins a running ring at the next epoch boundary from the lead's model and optimiser snapshot, fetched through the lead daemon, and the rendezvous hash moves about $1/n$ of the samples to it; this needs a script that uses a data loader. Training starts behind a barrier, so no rank trains while another still receives its closure.

E. Operating a Run

A run is meant to feel like running Python locally, so the client also stops, detaches and reports. An interrupt or a closed terminal sends one cancel frame; the executing daemon signals the job's process group and kills it two seconds later, and parallel chunks the job had asked for stop on every peer because each carries its request's context. A connection that ends without that frame never kills the job: the daemon logs every output frame in the job's directory, binds the lease to the job so that it outlives its expiry while the job runs, and lets the submitter detach and later reattach, replaying the log without gaps or repeats. Output is forwarded as raw chunks, so progress bars redraw. Each run ends with a summary of where its items ran, drawn from a receipt the shim prints at exit; a dry run builds the environment and reports unresolved imports, the parallel calls found in the source and the intended placement; and a per-machine events feed turns the daemon's log into plain-language lines. An offload flag keeps the script here but sends every distributed piece to other machines, and the summary shows what each machine spent.

IV. The Cost Model

Numerical work is gated. With b payload bytes, ϕ operations per byte, ρ link crossings, B bandwidth, R and R' local and remote throughput and K nodes, divisible work moves only when

$$\frac{b\phi}{R} \;>\; \frac{\rho\, b}{B} \;+\; \frac{b\phi}{R'K} \times 1.3. \tag{2}$$

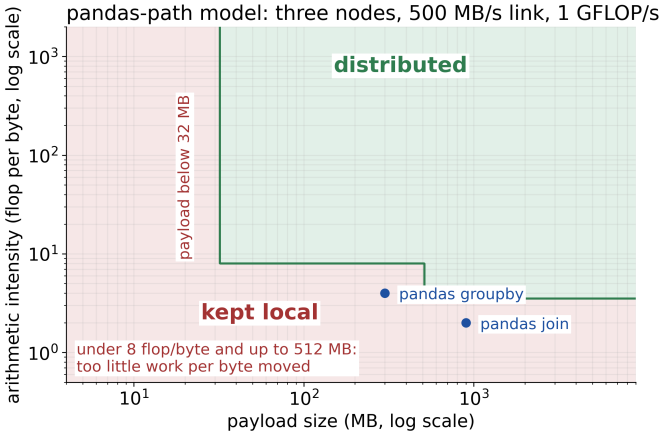


Fig. 2: Decision regions of Equation 2 for the pandas path ($\rho = 1$, 1 GFLOP/s) at three nodes on a 500 MB/s link.

Work needs at least two nodes and 32 MB. Work that does not fit in local memory moves regardless, and a matrix product that fits nowhere raises rather than exhausting the machine. A GPU peer raises R' sixfold. The inequality is first tried at an optimistic 10 Gbit/s and, only if offloading could win, bandwidth is measured with a 64 MB probe through the dispatch path. Indivisible work (SVD) moves when transfer costs under half the local time.

At about 200 GFLOP/s, local BLAS beats a CPU cluster's transport, so dense matrix products stay local; against a four-core peer local won by 25 to 100× and the model kept all four shapes local. An earlier version encoded "matrix multiplication never ships" as a rule; it was replaced by a check that the choice made was not the slower one, since the rule would be wrong on a cluster with a GPU.

V. Degradation Guarantee

Using the cluster must never change the result. Parallel-map chunks alternate between the local side and the cluster and fill a result table first-wins, idle local cores re-run any chunk not yet returned, and the call returns once the local side has covered every item, so an unresponsive cluster degrades to a local pool. Beneath that, every intercepted call falls back to the original on exception, a failed peer is skipped, a dead worker is respawned, and a failed job is rescheduled with back-off; a cancelled one is not retried, and a training ring continues on the ranks that are left. A mutation audit disables 95 features one at a time — among them gradient weighting, the rebalance hysteresis, roommate memory accounting and the router-mapping checks — and the suite fails for every one.

VI. Evaluation

A. Method and its limits

Two kinds of measurement are reported and kept apart. *Lab*: run for this paper on one 12-thread host with three worker daemons in containers; this measures correctness, overhead and caching, and **cannot** measure multi-machine speedup. *Recorded*: multi-machine figures measured by hand during development and recorded in the commit that each justified, on two machines over a home link or over Tailscale; they were not repeated for this paper. A third set comes from a rehearsal of the demonstration on a second host (16 threads, one laptop GPU), in containers shaped like a college laboratory; it too measures function and cost, not speedup.

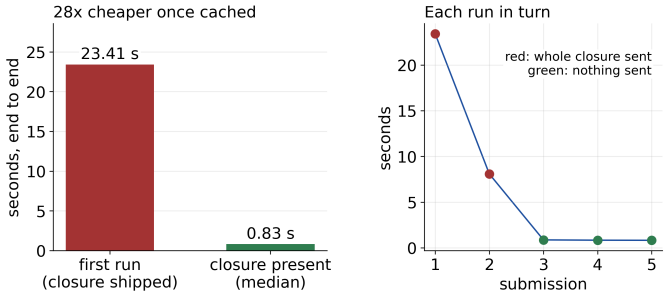


Fig. 3: Five submissions of the same environment in the container lab.

B. Lab results

Tests. 822 of **829** tests pass, **0** fail and **7** are skipped (two opt-in integration tests and five that depend on how Nix is installed on the host); before the latest round it was 615 of 617 with none failing. All 95 mutations are caught.

Environment cache. The first submission of a job took 23.4 s. The second landed on a worker that had not yet received the closure and took 8.1 s; the next three found the closure in place, shipped only the workspace, and took a median of 0.83 s (Fig. 3).

Interception cost. Start-up of a job that imports nothing heavy is 1.24× that without the shim, within the project's 3× budget (eager patching had cost 20×). On a whole job that the cost model declines, a 16-item map with 0.01 s of work, the median over five runs was 0.796 s with interception against 0.678 s without, every receipt showing nothing dispatched: interception that declines costs about 0.12 s per job (second host). An earlier whole-job comparison was not informative, because its workload had grown large enough to dispatch.

Distribution and failure. With distribution forced, 18 of 48 parallel-map items ran on three other daemons in every run, with identical checksums. The end-to-end fault-injection run, which kills a worker holding chunks, did not test recovery: on the freshly created lab the job ran on a node that already held its closure, the workers never received it, and the parallel map stayed local. We report it as a failed experiment; recovery is evidenced here only by tests in which every remote chunk fails. The cause was later traced to every worker refusing the broadcast closure for want of a trusted signing key (Section III-B).

Rehearsal. In the venue-shaped lab (containers on one laptop, so these are costs, not speedups), the headline parallel map took **43.5** s on its first run and **17.0** s cached; a GPU job submitted from a node without a GPU ran on the GPU node in **7.6** s with its environment cached (**264.8** s the first time); an interrupt stopped work on every peer within **2** s; a detached

TABLE I: Measurements between real machines, recorded during development

Measurement	Result
Pool map, 20-core host + 16-core peer, plain vs. ours	5.8–6.4 s vs. 4.9–5.4 s
Training, 1 machine / 2 ranks fp16 / 2 ranks int8	54.9 / 69.4 / 56.1 s
Training, final loss, same three	.087495 / .087497 / .087688
Ring of 16/8/2 cores, equal vs. measured batches	25.4 vs. 23.6 s
Closure diff, one package added	7 of 43 paths, −86% bytes
Peer reached with introducer suspended	0.03 s
Introducer traffic during verification	0.2 MB vs. ∼30 MB

TABLE II: PipedPeer against existing systems. The lower block is where it is the weaker system.

Property	Ours	Spark	Ray	Dask	ssh
What it does differently					
Runs unmodified Python	yes	no	part.	part.	yes
No cluster to configure	yes	no	no	no	part.
No head or master node	yes	no	no	no	yes
Reproduces the environment	yes	part.	part.	part.	no
Joins NATed machines, no relay	yes	no	no	no	no
Sandboxes each job	part.	no	no	no	no
Survives losing a node	yes	yes	yes	yes	no
Where it is behind					
Authenticated, encrypted	part.	yes	part.	part.	yes
Runtimes beyond Python	**no**	yes	part.	no	yes
Proven in production	**no**	yes	yes	yes	yes

job was reattached and returned its results; and in a second lab, two containers sharing one GPU, a two-rank ring finished its 80 epochs after one rank was killed, in **90.5** s.

C. Recorded multi-machine results

Table I lists them. They show modest gains and honest losses: a parallel map ran faster on two machines than on one, while two-machine training on a 54 Mbit/s link did not beat one machine (fp16 26% slower, int8 about level) — which the system itself reports at the end of each run.

D. Threats to validity

The lab shares one processor among all workers, so it cannot show speedup and distorts transfer costs; the rehearsal lab likewise cannot show wireless links, two physical GPUs or a campus network's rules. The recorded multi-machine figures were taken by hand, mostly once or a few times each, on whatever machines and links were at hand, and are evidence of behaviour rather than a controlled experiment; one single-sample figure in the project's history was later found to be wrong by a factor of nine on repetition, which is why figures from the project's repeatable benchmark scripts are preferred here wherever they exist.

VII. Conclusion

PipedPeer runs unmodified Python on other machines without configuring a cluster, by reproducing environments as content-addressed closures that are cached and diffed by store path, substituting the parallel primitives a script already uses, dividing work by measured speed, and joining machines across networks over direct, authenticated links with no relay. The most transferable lesson is the discipline of measuring and refusing: several features here exist in their current form because the first version measured slower than doing nothing.

Future work is led by closing the remaining security gaps — binding trusted signing keys to authenticated identities, requiring rather than preferring TLS, and authorisation finer than one shared secret — then a relay of last resort for peers no direct path can join, locality- and topology-aware placement, a parallel script and a training ring that survive their submitter and rank 0, and a controlled multi-machine evaluation.

References

[1] M. Zaharia, M. Chowdhury, M. J. Franklin, S. Shenker, and I. Stoica, "Spark: Cluster computing with working sets," in *Proc. 2nd USENIX Workshop on Hot Topics in Cloud Computing*, 2010.

[2] P. Moritz et al., "Ray: A distributed framework for emerging AI applications," in *Proc. 13th USENIX OSDI*, 2018, pp. 561–577.

[3] M. Rocklin, "Dask: Parallel computation with blocked algorithms and task scheduling," in *Proc. 14th Python in Science Conf.*, 2015, pp. 130–136.

[4] D. P. Anderson, "BOINC: A system for public-resource computing and storage," in *Proc. 5th IEEE/ACM Workshop on Grid Computing*, 2004, pp. 4–10.

[5] E. Dolstra, M. de Jonge, and E. Visser, "Nix: A safe and policy-free system for software deployment," in *Proc. 18th LISA*, 2004, pp. 79–92.

[6] Y. Deng, Z. Chen, X. Chen, and Y. Fang, "Task offloading in multi-hop relay-aided multi-access edge computing," *IEEE Trans. Veh. Technol.*, vol. 72, no. 1, pp. 1372–1376, 2023.

[7] D. Yong, R. Liu, X. Jia, and Y. Gu, "Joint optimization of multi-user partial offloading strategy and resource allocation in D2D-enabled MEC," *Sensors*, vol. 23, no. 5, 2023.

[8] H. He, X. Yang, X. Mi, H. Shen, and X. Liao, "Multi-agent deep reinforcement learning based dynamic task offloading in a D2D mobile-edge computing network," *Sensors*, vol. 24, no. 16, 2024.

[9] Z. Zhao, J. Perazzone, G. Verma, and S. Segarra, "Congestion-aware distributed task offloading in wireless multi-hop networks using graph neural networks," arXiv:2312.02471, 2023.

[10] D. Petersohn et al., "Towards scalable dataframe systems," *Proc. VLDB Endowment*, vol. 13, no. 12, pp. 2033–2046, 2020.

[11] A. Bergström, "Programming models for failure-transparent distributed systems," M.S. thesis, KTH Royal Institute of Technology, 2025.

[12] J. Iyengar and M. Thomson, "QUIC: A UDP-based multiplexed and secure transport," RFC 9000, IETF, May 2021.

[13] M. Petit-Huguenin, G. Salgueiro, J. Rosenberg, D. Wing, R. Mahy, and P. Matthews, "Session Traversal Utilities for NAT (STUN)," RFC 8489, IETF, Feb. 2020.

[14] S. Cheshire and M. Krochmal, "NAT Port Mapping Protocol (NAT-PMP)," RFC 6886, IETF, Apr. 2013.

[15] D. Wing, S. Cheshire, M. Boucadair, R. Penno, and P. Selkirk, "Port Control Protocol (PCP)," RFC 6887, IETF, Apr. 2013.

[16] B. Ford, P. Srisuresh, and D. Kegel, "Peer-to-peer communication across network address translators," in *Proc. USENIX Annual Technical Conf.*, 2005, pp. 179–192.

[17] E. Rescorla, "Keying material exporters for Transport Layer Security (TLS)," RFC 5705, IETF, Mar. 2010.

[18] D. G. Thaler and C. V. Ravishankar, "Using name-based mappings to increase hit rates," *IEEE/ACM Trans. Netw.*, vol. 6, no. 1, pp. 1–14, Feb. 1998.